AI Large Model Offline Voice Assistant

Note: Raspberry Pi 5 1GB has no large model package and cannot run it. 2GB and 4GB versions cannot run offline due to performance limitations. You can refer to the online large model tutorial.

1. Offline Voice Configuration

Before setting up auto-start, we must ensure that the program itself can work independently in offline mode. This requires modifying the configuration file.

1. **Locate the configuration file**:
   In your project code, find and open the configuration file:
   `config/yahboom.yaml`
2. **Modify configuration parameters**:
   Please check the following parameters in the file and ensure their values match what's shown below. If a parameter doesn't exist, add it.

```
asr:                                  #Voice node parameters
  ros__parameters:
    VAD_MODE: 2                       #VAD sensitivity
    sample_rate: 16000                #ASR recording audio sample rate
    frame_duration_ms:  30           #VAD frame size in ms
    use_oline_asr: False             #Whether to use online ASR recognition (True
for online, False for offline)
    mic_serial_port: "/dev/ttyUSB0"   #Microphone serial port alias
    mic_index: 0                      #Microphone index
    language: 'zh'                    #ASR language
    regional_setting : "China"        #international: International version, China:
China version

model_service:                        #Model server node parameters
  ros__parameters:
    language: 'zh'                    #Large model interface language
    useolinetts: False                 #Whether to use online TTS (True for online,
False for offline)

     # Large model configuration
```

```
    # llm_platform: 'ollama'              # Optional platforms: 'ollama', 'tongyi',
'spark', 'qianfan', 'openrouter'
    llm_platform: 'ollama'              # Currently selected large model platform
    regional_setting : "China"
```

- `use_oline_asr` and `useolinetts` must be set to `False`.
- `llm_platform` must be set to `ollama`.

  With these settings, everything will be offline.

3. **Save the file** and **rebuild** the project to apply changes:

```
cd ~/yahboom_ws
colcon build
source install/setup.bash
```

After completing this step, the program is now a pure offline voice service.

2. Create Auto-Start Service (Systemd)

Now, we will create a `systemd` service to make `largemodel_control.launch.py` run automatically at system startup.

Note: If you are using Yahboom's AI large model factory image or SSD factory image, these scripts have already been created. You only need to restart the auto-start container.

2.1 Create a Docker Container Startup Script

1. **Create the script file**:
   In the home directory (`~`), create a file named `ros2_docker_autostart.sh`.

```
vim ~/ros2_docker_autostart.sh
```

2. **Write script content**:
   Copy and paste the following content into the script file.

```
#!/usr/bin/env bash

set -e

CN=${CONTAINER_NAME:-ros_humble_ai_service}
IMG=${IMAGE_NAME:-ros_humble_ai:v1.0}
WS=${HOST_WS_DIR:-"$HOME/yahboom_ws"}
ROS=${ROS_DISTRO:-humble}
CMD=${LAUNCH_CMD:-"ros2 launch largemodel largemodel_control.launch.py"}
AUID=${AUDIO_UID:-$(id -u)}
XAUTH=${XAUTHORITY:-"$HOME/.Xauthority"}

mkdir -p "$WS"

x=()
```

```
if [ -d /tmp/.X11-unix ]; then
  command -v xhost >/dev/null 2>&1 && xhost +local:root || true
  x+=( -e DISPLAY=${DISPLAY:-:0} -e QT_X11_NO_MITSHM=1 -v /tmp/.X11-unix:/tmp/.X11-
unix )
  [ -f "$XAUTH" ] && x+=( -v "$XAUTH:/root/.Xauthority" -e
XAUTHORITY=/root/.Xauthority )
fi

a=()
if [ -S "/run/user/$AUID/pulse/native" ]; then
  a+=( -e PULSE_SERVER=unix:/run/user/$AUID/pulse/native -v
"/run/user/$AUID/pulse:/run/user/$AUID/pulse:ro" )
  [ -d "$HOME/.config/pulse" ] && a+=( -v "$HOME/.config/pulse:/root/.config/pulse:ro"
)
fi

d=()
[ -e /dev/video0 ] && d+=( --device=/dev/video0 )
[ -d /dev/bus/usb ] && d+=( --device=/dev/bus/usb )
[ -d /dev/snd ] && d+=( --device=/dev/snd )
[ -e /dev/mic ] && d+=( -v /dev/mic:/dev/mic )

if docker ps -a --format '{{.Names}}' | grep -qx "$CN"; then
  docker ps --format '{{.Names}}' | grep -qx "$CN" || docker start "$CN" >/dev/null
else
  docker run -d \
    --name "$CN" --net=host --privileged --restart unless-stopped \
    --security-opt apparmor:unconfined \
    -v "$WS:/root/yahboom_ws:rw" \
    "${d[@]}" "${x[@]}" "${a[@]}" \
    "$IMG" \
    bash -lc "source /opt/ros/$ROS/setup.bash; [ -f
/root/yahboom_ws/install/setup.bash ] && source /root/yahboom_ws/install/setup.bash;
$CMD"
fi

echo "OK: $CN"
```

3. **Save and exit**

4. Enable Docker to start on boot:

```
sudo systemctl enable --now docker
```

5. **Grant script execution permission**, then run:

```
chmod +x ~/ros2_docker_autostart.sh
./ros2_docker_autostart.sh
```

6. Verify:

```
docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' ros_humble_ai_service
```

The output should be `unless-stopped`. After this, Docker will automatically pull up the container each time you boot, achieving the effect of auto-starting the voice assistant.

2.2 If using Yahboom's AI large model factory image or SSD factory image, you don't need to recreate the script. Just run the following command to start the auto-start service:

```
~/ros2_docker_autostart.sh
```

2.3 Related Commands:

Immediately stop the container, which will also stop auto-start:

```
docker stop ros_humble_ai_service
```

View Docker container running logs:

```
docker logs -f ros_humble_ai_service
```

3. CSI Camera Auto-Start Service

To call CSI video in Docker, you need to start host_stream.py on the host machine.
Therefore, if you need to call CSI camera video during auto-start, you also need to add this file to the Systemd auto-start service.

Note: If you are using Yahboom's AI large model factory image or SSD factory image, these scripts have already been created. You only need to restart the auto-start service.

1. Enter the command to write the file:

```
sudo vim /etc/systemd/system/host-stream.service
```

2. Copy the following content into the file:

```
[Unit]
Description=Host stream (runs ~/host_stream.py)
After=network.target

[Service]
```

```
Type=simple
User=pi
WorkingDirectory=/home/pi
ExecStart=/usr/bin/python3 /home/pi/host_stream.py
Restart=on-failure
RestartSec=5
StandardOutput=append:/home/pi/host_stream.log
StandardError=append:/home/pi/host_stream.log

[Install]
WantedBy=multi-user.target
```

3. Save and exit
4. Reload and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable host-stream.service
sudo systemctl start host-stream.service
```

5. Check service status:

```
sudo systemctl status host-stream.service

If you see `Active: active (running)`, the service has started successfully!
```

6. When you don't need the CSI camera auto-start service, run the following command to stop:

```
sudo systemctl stop host-stream.service
sudo systemctl disable host-stream.service
```

If using Yahboom's AI large model factory image or SSD factory image, you don't need to recreate the script. Just run the following command to restart the auto-start service:

```
sudo systemctl enable host-stream.service
sudo systemctl start host-stream.service
```